# PROPOSAL ĐỒ ÁN

## KIẾN TRÚC & TỔ CHỨC DỮ LIỆU

# Hệ Thống Quản Lý Thư Viện

Library Management System — Proposal Kiến Trúc & Dữ Liệu

Phiên bản 1.0

## Contents


Yêu cầu chung ..... 2
1. Kiến Trúc Phần Mềm ..... 5
2. Tổ Chức Dữ Liệu ..... 5
  2.1 Tập Dữ Liệu Người Dùng (Độc Giả) ..... 5
  2.2 Tập Dữ Liệu Tài Liệu (Catalog) ..... 6
  2.3 Dữ Liệu Mượn/Trả & Vận Hành ..... 6
  2.4 Neo4j Knowledge Graph — Mạng Chủ Đề ..... 7
3. Tính Năng ..... 7
  3.1 Intelligent Search & Discovery ..... 7
  3.2 Personalized Reading Journey ..... 7
  3.3 Librarian AI Assistant ..... 8
4. Dữ Liệu Thực ..... 8

# Yêu cầu chung

**1. Giới thiệu tổng quan**

- **Tên đồ án:** Hệ thống quản lý thư viện
- **Mục tiêu:** Xây dựng một phần mềm quản lý toàn bộ các nghiệp vụ của một thư viện, từ việc quản lý sách, độc giả, đến việc mượn/trả sách. Hệ thống giúp tự động hóa quy trình, giảm thiểu sai sót và  hiệu quả hoạt động của thư viện.
- **Đối tượng sử dụng:** Thủ thư (người quản lý) và độc giả (người dùng).

---

**2. Yêu cầu chi tiết**

Dưới đây là các chức năng cần có trong hệ thống. Bạn có thể chọn triển khai một số hoặc tất cả các chức năng này tùy theo phạm vi của đồ án.

**2.1. Phân hệ Thủ thư (Quản lý)**

Phân hệ này dành cho những người có quyền quản lý cao nhất trong hệ thống.

- **Quản lý tài liệu (sách):**
    - Thêm mới, sửa, xóa thông tin sách (tên sách, tác giả, nhà xuất bản, năm xuất bản, thể loại, số lượng).
    - Quản lý tình trạng sách (còn/hết, đang được mượn, hỏng).
    - Hỗ trợ nhập liệu số lượng lớn qua file Excel hoặc CSV.
- **Quản lý độc giả:**
    - Thêm mới, sửa, xóa thông tin độc giả (tên, mã số sinh viên/mã độc giả, ngày sinh, địa chỉ, số điện thoại).
    - Quản lý danh sách thẻ độc giả.
- **Quản lý mượn/trả:**
    - Lập phiếu mượn sách.
    - Kiểm tra tình trạng sách và độc giả trước khi cho mượn.
    - Ghi nhận thông tin trả sách.
    - Xử lý các trường hợp sách trả muộn hoặc làm hỏng/mất sách (tính tiền phạt).
- **Thống kê và báo cáo:**
    - Thống kê số lượng sách theo thể loại, tác giả.
    - Báo cáo sách được mượn nhiều nhất.
    - Thống kê độc giả mượn sách nhiều nhất.
    - Báo cáo độc giả quá hạn trả sách.

**2.2. Phân hệ độc giả (Người dùng)**

Phân hệ này dành cho người dùng cuối là các độc giả.

- **Tra cứu và tìm kiếm:**
    - Tra cứu sách theo tên, tác giả, nhà xuất bản, thể loại.

- Xem thông tin chi tiết của sách (số lượng còn lại, mô tả).

- **Quản lý thông tin cá nhân:**
  - Đăng ký và đăng nhập hệ thống.
  - Xem thông tin thẻ độc giả và lịch sử mượn/trả sách.
  - Xem danh sách sách đang mượn và hạn trả.
- **Đặt sách:**
  - Đặt trước những cuốn sách đang được mượn.
  - Nhận thông báo khi sách đã có sẵn.

---

**3. Yêu cầu kỹ thuật**

- **Nền tảng:** Ứng dụng web, desktop, hoặc mobile (tùy chọn). Thường sẽ là ứng dụng web để dễ dàng truy cập.
- **Ngôn ngữ lập trình:**
  - **Backend:** Python (Django/Flask), Java (Spring Boot), Node.js (Express), PHP (Laravel).
  - **Frontend:** HTML, CSS, JavaScript (ReactJS, VueJS, Angular).
- **Cơ sở dữ liệu:**
  - **MySQL**, **PostgreSQL** là những lựa chọn phù hợp cho dự án này vì dữ liệu có cấu trúc rõ ràng.
- **Kiến trúc:**
  - Sử dụng kiến trúc client-server.
  - Giao tiếp giữa frontend và backend qua API RESTful.

---

**4. Hướng dẫn chi tiết thực hiện**

**Bước 1: Phân tích và thiết kế (1-2 tuần)**

1. **Phân tích yêu cầu:** Ghi lại tất cả các chức năng chính và chức năng phụ của hệ thống.
2. **Thiết kế cơ sở dữ liệu (ERD):** Vẽ sơ đồ quan hệ thực thể (ERD) cho các bảng chính như Sach, DocGia, PhieuMuon, PhieuTra, NhaXuatBan, TacGia. Điều này giúp bạn có cái nhìn tổng quan về cách dữ liệu được lưu trữ và liên kết.
3. **Thiết kế giao diện (UI/UX):**
   - Vẽ sơ đồ luồng người dùng để hình dung cách độc giả và thủ thư tương tác với hệ thống.
   - Sử dụng các công cụ như **Figma** hoặc **draw.io** để thiết kế giao diện (wireframe và mockup) cho cả hai phân hệ.

**Bước 2: Lựa chọn công nghệ (1 tuần)**

- Dựa trên kiến thức và sở thích, bạn hãy chọn một bộ công nghệ phù hợp.

- **Ví dụ:** Bạn có thể chọn **MERN Stack** (MongoDB, ExpressJS, ReactJS, NodeJS) hoặc **LAMP Stack** (Linux, Apache, MySQL, PHP).

**Bước 3: Lập trình và kiểm thử (4-6 tuần)**

1. **Lập trình Backend:**
    - Thiết lập môi trường phát triển (tạo dự án, cài đặt thư viện).
    - Xây dựng API cho từng chức năng đã thiết kế ở Bước 1.
    - Kết nối với cơ sở dữ liệu.
2. **Lập trình Frontend:**
    - Xây dựng giao diện người dùng dựa trên thiết kế.
    - Gửi các request đến API backend để lấy và xử lý dữ liệu.
3. **Kiểm thử:**
    - **Kiểm thử đơn vị (Unit Test):** Kiểm tra từng chức năng nhỏ.
    - **Kiểm thử tích hợp (Integration Test):** Kiểm tra sự kết nối giữa frontend và backend, đảm bảo dữ liệu được truyền đi và nhận về đúng cách.
    - **Kiểm thử hệ thống (System Test):** Giả lập một người dùng thực tế để kiểm tra toàn bộ luồng công việc.

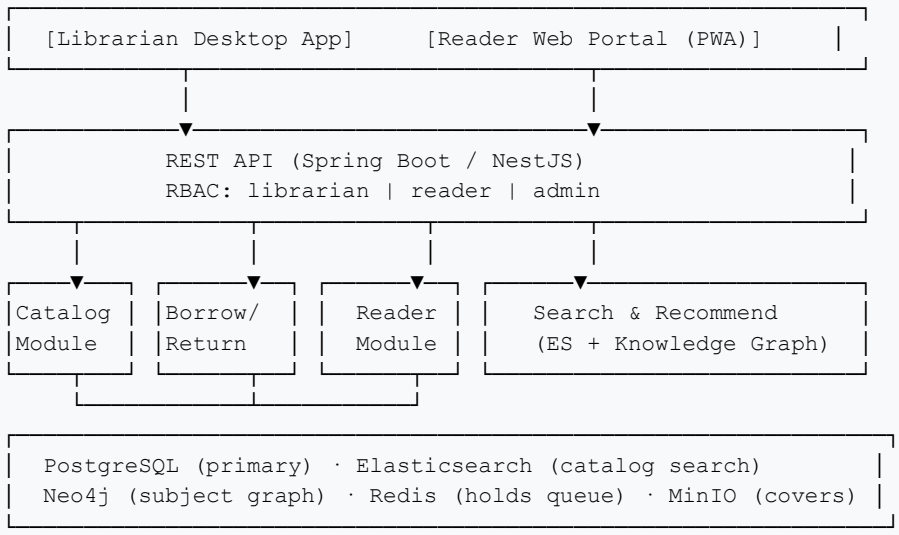
**Bước 4: Hoàn thiện và báo cáo (1 tuần)**

1. **Hoàn thiện sản phẩm:** Sửa các lỗi còn tồn đọng, tối ưu hóa hiệu suất và giao diện.
2. **Viết báo cáo:**
    - Nêu rõ mục tiêu, phạm vi và công nghệ đã sử dụng.
    - Mô tả chi tiết kiến trúc hệ thống và luồng xử lý của các chức năng.
    - Đưa ra các hình ảnh minh họa về giao diện và sơ đồ kỹ thuật.
3. **Chuẩn bị bản trình bày:**
    - Chuẩn bị slide trình chiếu.
    - Thực hiện demo sản phẩm trực tiếp.

# 1. Kiến Trúc Phần Mềm

Hệ thống thư viện áp dụng kiến trúc Monolith với domain separation rõ ràng. Điểm đặc biệt: tích hợp Elasticsearch cho catalog tra cứu (full-text, ISBN, metadata) và Neo4j Graph DB cho mạng quan hệ chủ đề sách phục vụ recommendation.

**Kiến Trúc — Library Management System**




# 2. Tổ Chức Dữ Liệu

## 2.1 Tập Dữ Liệu Người Dùng (Độc Giả)

**Schema — Reader Profile**

```
TABLE: readers
  reader_id       UUID PK
  user_id         UUID FK → auth.users
  reader_code     VARCHAR(20) UNIQUE  -- 'LIB-2024-00001'
  full_name       VARCHAR(200)
  birth_date      DATE
  phone           VARCHAR(20)
  email           VARCHAR(255)
  reader_type     ENUM(student,lecturer,staff,public)
  faculty         VARCHAR(200)  -- nếu là SV/GV
  card_issued     DATE
  card_expired    DATE
  status          ENUM(active, suspended, expired)
  max_books       SMALLINT DEFAULT 5  -- theo reader_type
  avatar_url      TEXT
  created_at      TIMESTAMPTZ

TABLE: reader_preferences  -- cá nhân hóa
  reader_id          UUID FK PK
  preferred_subjects UUID[]  -- FK → subjects
  preferred_authors  TEXT[]
  preferred_langs    VARCHAR[]
  reading_pace       ENUM(slow, medium, fast)  -- từ borrow history
  notification_pref  JSONB  -- { email: true, sms: false, app: true }
```

## 2.2 Tập Dữ Liệu Tài Liệu (Catalog)

### Schema — Bibliographic Catalog

```
TABLE: bibliographic_records  -- thông tin thư mục (Dublin Core)
  bib_id          UUID PK
  isbn            VARCHAR(20)
  title           VARCHAR(500) NOT NULL
  subtitle        VARCHAR(500)
  authors         JSONB  -- [{ name, role: 'author/editor/translator' }]
  publisher_id    UUID FK → publishers
  publish_year    SMALLINT
  edition         VARCHAR(50)
  language        VARCHAR(10)
  description     TEXT  -- abstract / tóm tắt
  page_count      INT
  call_number     VARCHAR(50)  -- DDC: '005.133'
  ddc_class       VARCHAR(20)  -- Dewey Decimal Classification
  subject_headings TEXT[]  -- LCSH subject headings
  keywords        TEXT[]
  cover_url       TEXT
  metadata        JSONB  -- { size, weight, format, series, ... }
  created_at      TIMESTAMPTZ

TABLE: book_copies  -- từng bản vật lý
  copy_id          UUID PK
  bib_id           UUID FK → bibliographic_records
  barcode          VARCHAR(50) UNIQUE
  condition        ENUM(new, good, fair, poor, damaged)
  location_id      UUID FK → shelf_locations
  status           ENUM(available, borrowed, reserved, lost, processing)
  acquired_date    DATE
  acquired_price   NUMERIC(12,2)

TABLE: shelf_locations
  location_id UUID PK
  floor       SMALLINT
  section     VARCHAR(20)  -- 'A','B','C'
  shelf       VARCHAR(10), position VARCHAR(10)
  ddc_range   VARCHAR(50)  -- '000-099 Tin học'
```

## 2.3 Dữ Liệu Mượn/Trả & Vận Hành

### Schema — Circulation

```
TABLE: borrow_transactions
  tx_id          UUID PK
  reader_id      UUID FK
  copy_id        UUID FK
  borrow_date    DATE NOT NULL
  due_date       DATE NOT NULL
  return_date    DATE  -- NULL = chưa trả
  renewed_count  SMALLINT DEFAULT 0
  status         ENUM(borrowed, returned, overdue, lost)
  fine_amount    NUMERIC(10,2) DEFAULT 0
  fine_paid      BOOLEAN DEFAULT FALSE
  issued_by      UUID FK → staff
  returned_to    UUID FK → staff
  created_at     TIMESTAMPTZ
```

```
TABLE: holds  -- đặt giữ sách
  hold_id       UUID PK
  reader_id     UUID FK
  bib_id        UUID FK  -- hold theo đầu sách
  requested_at  TIMESTAMPTZ
  notified_at   TIMESTAMPTZ  -- khi có copy sẵn
  expires_at    TIMESTAMPTZ  -- hết hạn giữ
  status        ENUM(waiting, notified, fulfilled, cancelled, expired)
  queue_position INT  -- vị trí trong hàng chờ
```

### 2.4 Neo4j Knowledge Graph — Mạng Chủ Đề

**Neo4j Knowledge Graph Design**

```
-- Graph schema trong Neo4j
Node types:
  (:Book { bib_id, title, isbn })
  (:Subject { name, ddc_code })
  (:Author { name, id })
  (:Reader { reader_id })

Relationship types:
  (:Book)-[:ABOUT { relevance: 0.0-1.0 }]→(:Subject)
  (:Book)-[:WRITTEN_BY { role }]→(:Author)
  (:Author)-[:SIMILAR_TO { score }]→(:Author)
  (:Subject)-[:RELATED_TO { weight }]→(:Subject)
  (:Reader)-[:BORROWED { times, last_date }]→(:Book)
  (:Reader)-[:INTERESTED_IN]→(:Subject)

-- Query ví dụ: 'Sách liên quan đến cuốn X'
MATCH (b:Book {bib_id: $id})-[:ABOUT]->(s:Subject)
      <-[:ABOUT]-(related:Book)
WHERE related.bib_id <> $id
RETURN related ORDER BY count(s) DESC LIMIT 10
```

## 3. Tính Năng

### 3.1 Intelligent Search & Discovery

- Full-text search với Elasticsearch: title, author, subject heading, abstract
- ISBN/barcode scan → instant book lookup (mobile PWA camera)
- Faceted search: ngôn ngữ, năm xuất bản, DDC class, availability
- 'Sách liên quan' powered by Neo4j graph traversal theo subject headings
- Semantic search: tìm theo chủ đề tự nhiên 'sách về trí tuệ nhân tạo cho người mới'

### 3.2 Personalized Reading Journey

- Reading list cá nhân: bookmark, reading progress, notes trên đầu sách
- 'Tiếp theo nên đọc gì?': recommendation từ Neo4j + collaborative filtering
- Dynamic loan period: độc giả đọc nhanh (fast reader) có thể gia hạn ít hơn
- Proactive notification: nhắc trả sách 2 ngày trước hạn, thông báo sách hold sẵn

### 3.3 Librarian AI Assistant

- 'Còn bao nhiêu cuốn sách về Machine Learning?' → catalog query
- 'Độc giả nào đang giữ 'Đắc Nhân Tâm'?' → borrow transaction lookup
- 'Gợi ý mua sách mới theo nhu cầu tháng qua' → analyze borrow trends + holds

## 4. Dữ Liệu Thực

| Bảng | Target | Ghi chú |
|---|---|---|
| bibliographic_records | 10,000+ | Import MARC records từ thư viện quốc gia |
| book_copies | 30,000+ | Trung bình 3 bản/đầu sách |
| readers | 5,000+ | Sinh viên + giảng viên + cán bộ |
| borrow_transactions | 100,000+ | 3 năm lịch sử giao dịch |
| Neo4j nodes | 15,000+ | Books + Subjects + Authors |